Machine Learning

Neural Networks and Back-Propagation

Dong-Kyu Chae

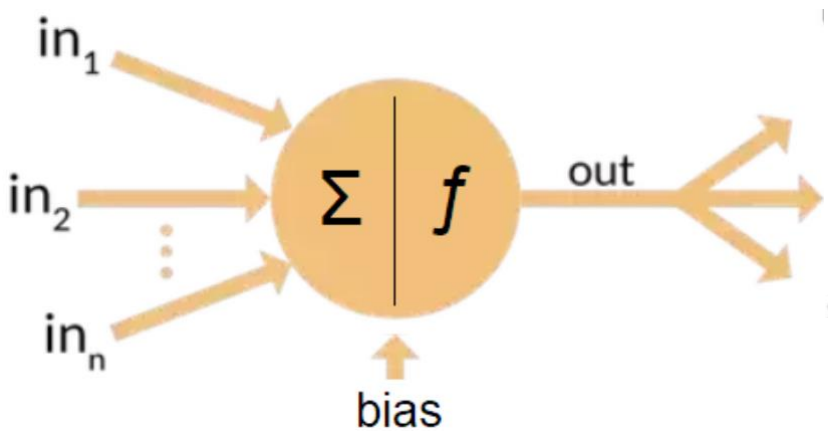
PI of the Data Intelligence Lab @HYU
Department of Computer Science
Hanyang University

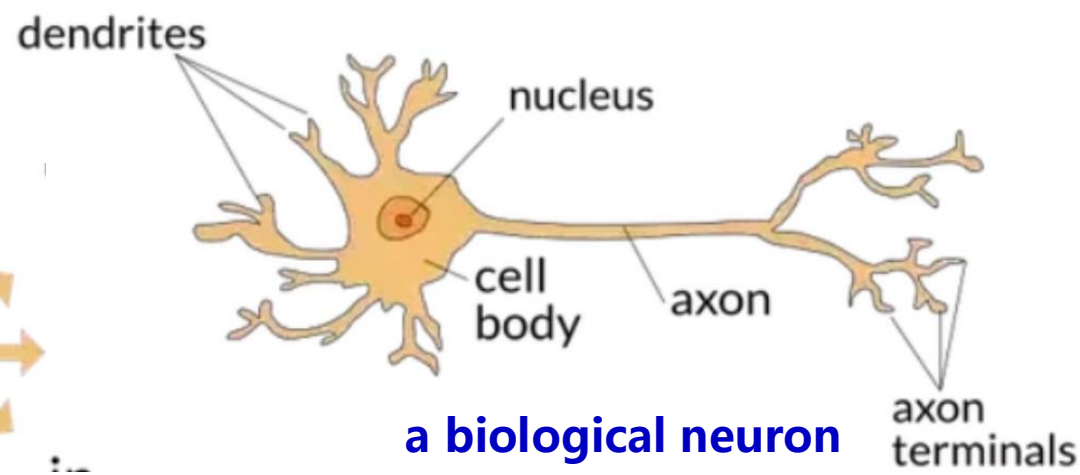
Data
Intelligence
LAB

Introduction

- **(Artificial) neural networks**
  - https://en.wikipedia.org/wiki/Artificial_neural_network
  - Artificial neural networks (ANNs), usually simply called **neural networks** (NNs), are computing systems vaguely inspired by the **biological neural networks that constitute animal brains**.
  - **Artificial neurons** are elementary units in an artificial neural network.

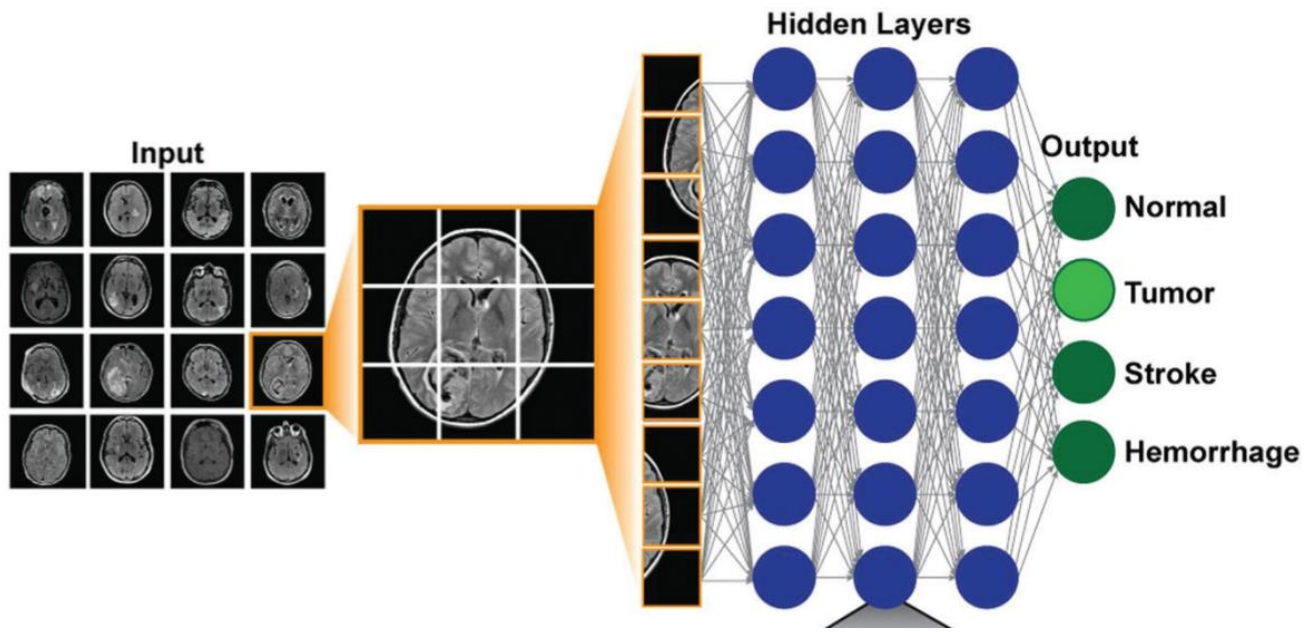
an artificial neuron



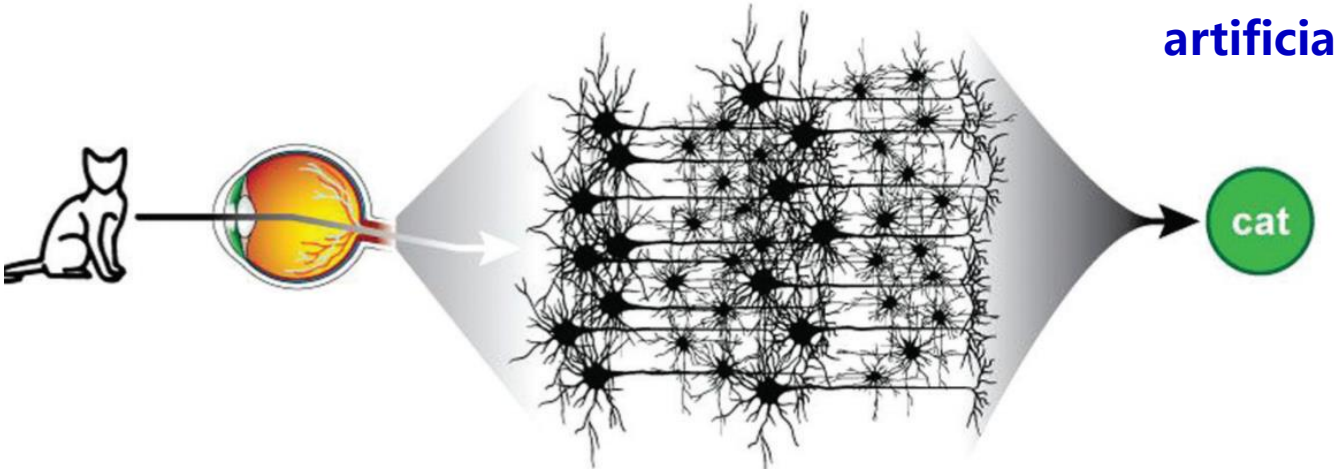


a biological neuron

Introduction



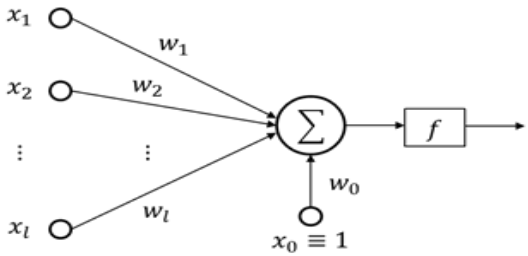
artificial neural networks



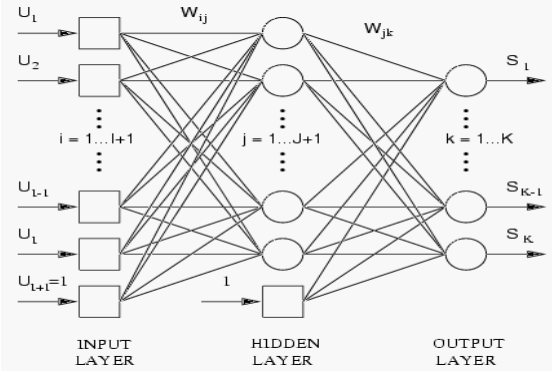
biological neural networks

Neural Network

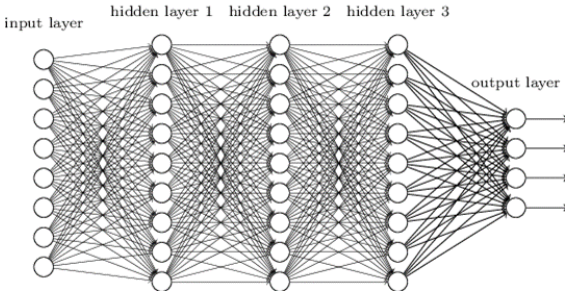
- **History**
  - A neuron (or a perceptron) (1958~)



  - Multi-layer neural network (1986~)

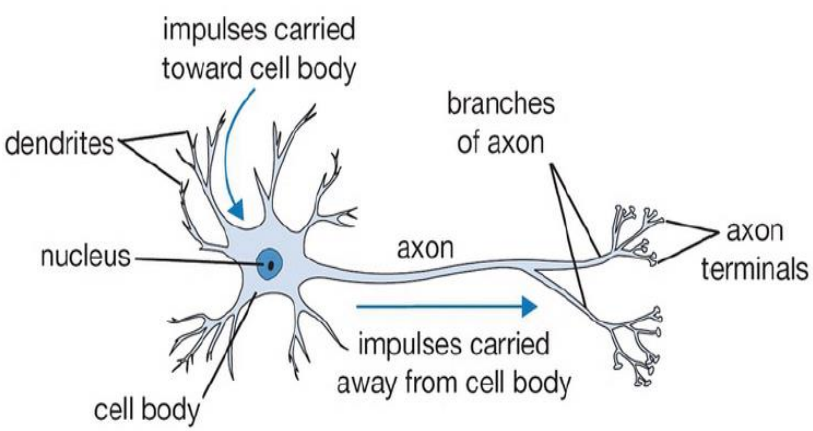


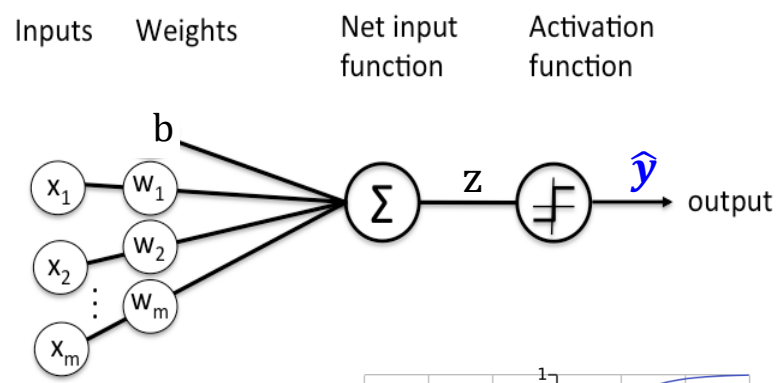
  - Deep learning (2006~)



A Neuron (Perceptron)

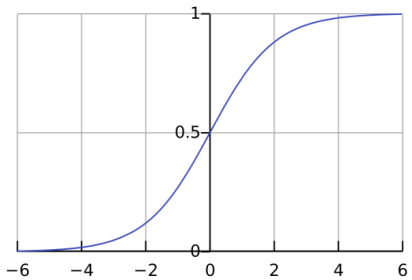
- **Artificial neuron (or just neuron, or perceptron)**
  - It is a model inspired by biological neuron
  - Composed of **(1) weighted sum of inputs** and **(2) an activation function** given the summated input





$$z = \sum_{i=1}^{n} x_i w_i + \mathrm{b}$$

$$\hat{y} = \sigma(z)$$



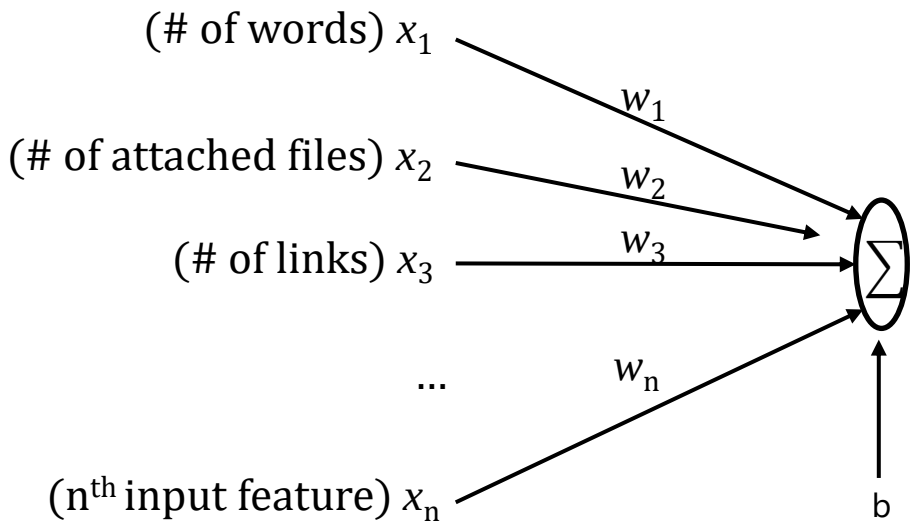
$\sigma(z)$: a sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

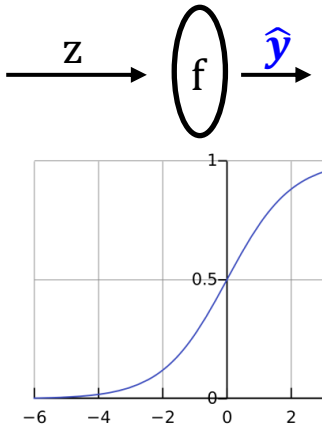
A Neuron (Perceptron)

- **Example**
  - **Input data**: emails
  - **Output**: spam/normal
    - **spam**: 1
    - **normal**: 0

	# of words	# of attached files	# of links	# of malicious words	.....	spam
mail #1	256	0	3	7	.....	1 (Yes)
mail #2	56	1	0	3	.....	0 (No)
mail #3	24	1	0	1	.....	0
mail #4	672	0	0	0	.....	0
mail #5	67	2	4	3	.....	1
mail #6	48	0	2	6	.....	0
mail #7	79	1	3	8	.....	1
.....						



$$z = \sum_{i=1}^{n} x_i w_i + \mathrm{b}$$



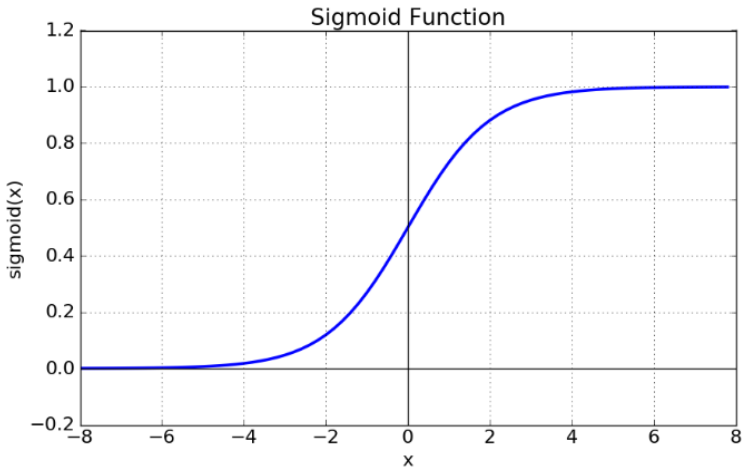
$$\hat{y} = \sigma(z)$$

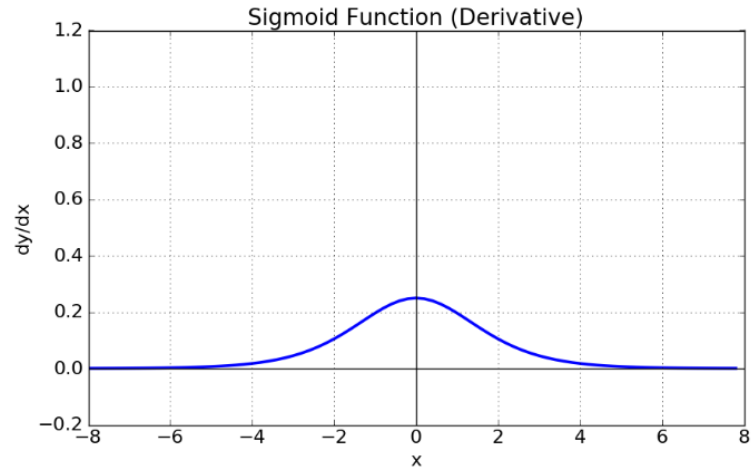
1 (spam, if $\hat{y} > 0.5$)
0 (normal, if $\hat{y} \leq 0.5$)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Appendix

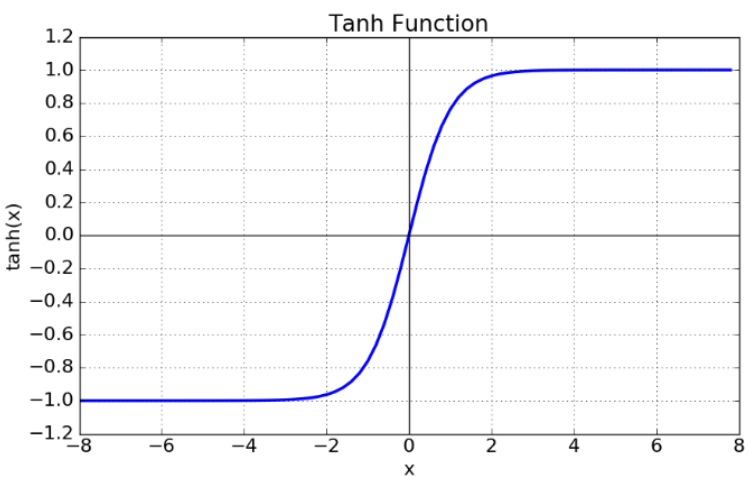
Activation functions

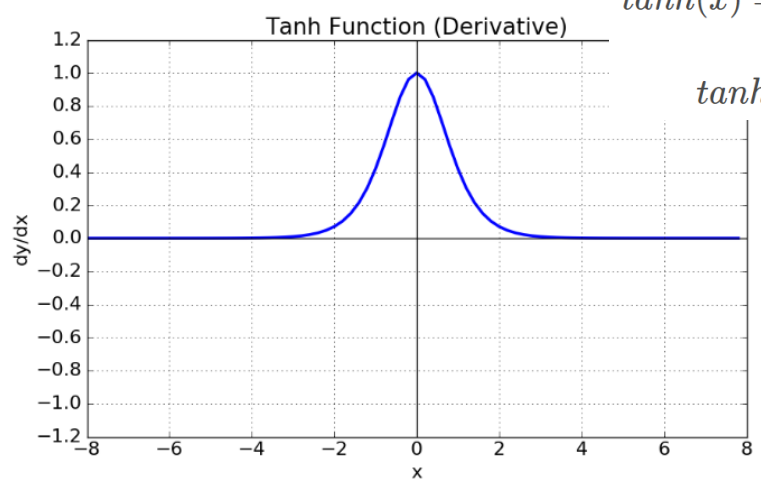




$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\sigma'(x) = \sigma(x)(1-\sigma(x))$$





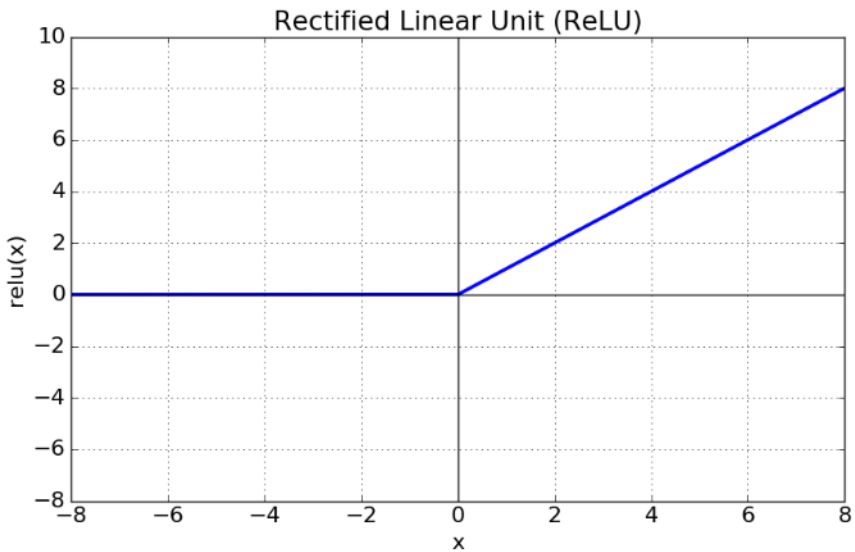
$$tanh(x) = 2\sigma(2x) - 1 = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$tanh'(x) = 1 - tanh^2(x)$$

Low gradient values
=> slow training

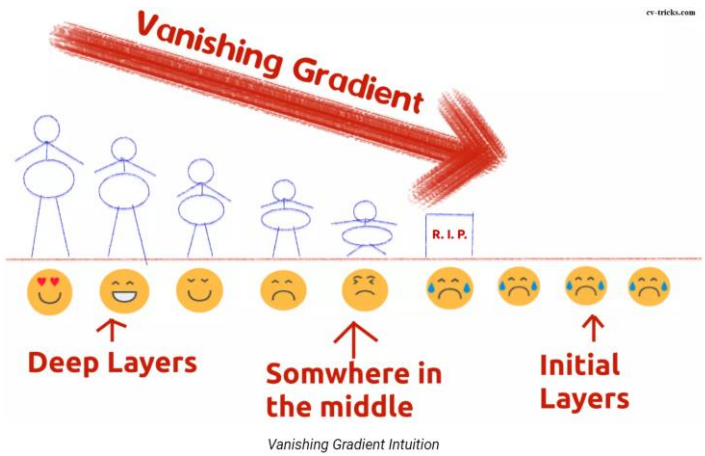
Appendix

Activation functions



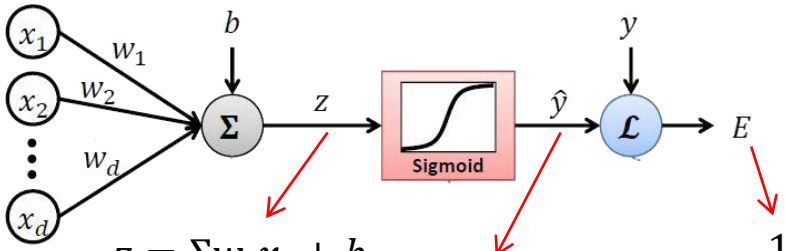
$$f(x) = max(0, x)$$

- The ReLU activation function helps solve the gradient-vanishing problem!



A Neuron (Perceptron)

- **Cost function** $E = \frac{1}{2n}\sum_{k=1}^{n}(y_k - \hat{y}_k)^2$**, where** $\hat{y}_k = \frac{1}{1+e^{-z_k}}$ **and** $z_k = \sum_{i=1}^{d} x_{ki}w_i + b$
- **Goal: w*,*b** =** $\underset{\mathbf{w},b}{\operatorname{argmin}}\, E$
- **Solution: gradient descent**
  - Initialize **w,** *b*
  - Repeat
    - $w_i^{t+1} = w_i^t - \mu \frac{\partial E}{\partial w_i}$ for each i=1~d,
    - $b_i^{t+1} = b_i^t - \mu \frac{\partial E}{\partial b}$



$z = \Sigma w_i x_i + b$

$\hat{y} = \frac{1}{1+e^{-z}}$

$E = \frac{1}{2}(y - \hat{y})^2$

(simplified for just one data point)

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial w_i} = \frac{\partial E}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z}\frac{\partial z}{\partial w_i}$$

$$= -(y - \hat{y})\hat{y}(1 - \hat{y})x_i$$

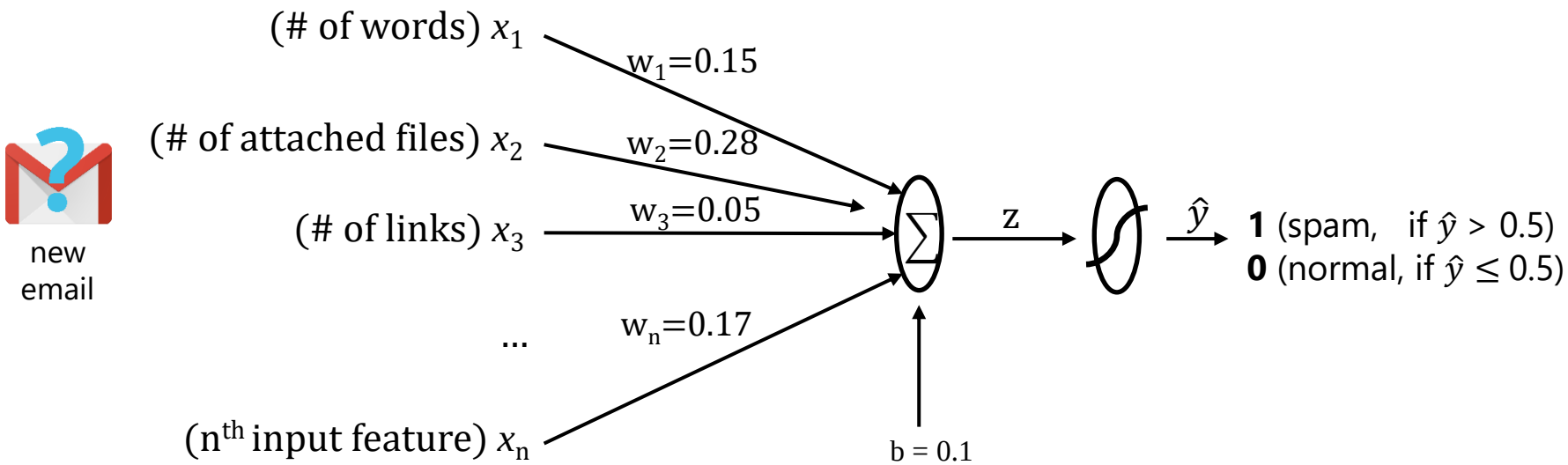
$$\frac{\partial E}{\partial b} = \frac{\partial E}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z}\frac{\partial z}{\partial b} = -(y - \hat{y})\hat{y}(1 - \hat{y})$$

$$\frac{\partial z}{\partial w_i} = x_i \qquad \frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}) \qquad \frac{\partial E}{\partial \hat{y}} = -(y - \hat{y})$$

$$\frac{\partial z}{\partial b} = 1$$

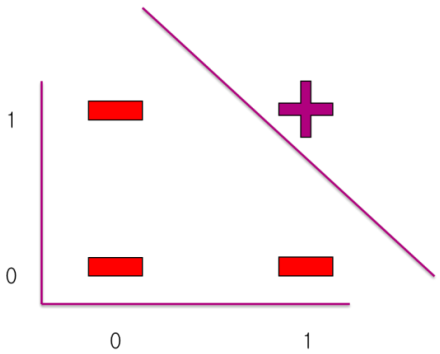
A Neuron (Perceptron)

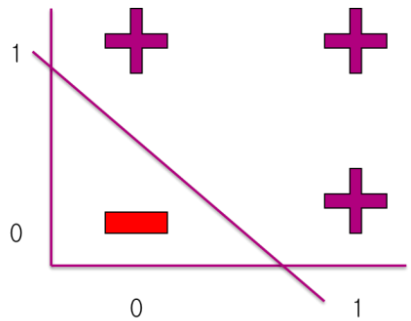
Classification (after training: testing)



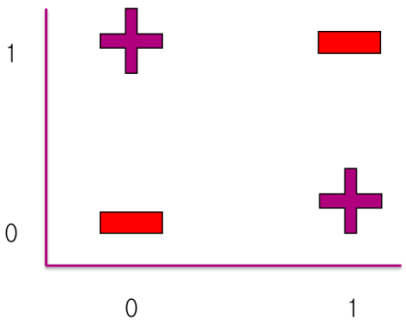
A Neuron (Perceptron)

- **Limitation of a neuron: linearity**



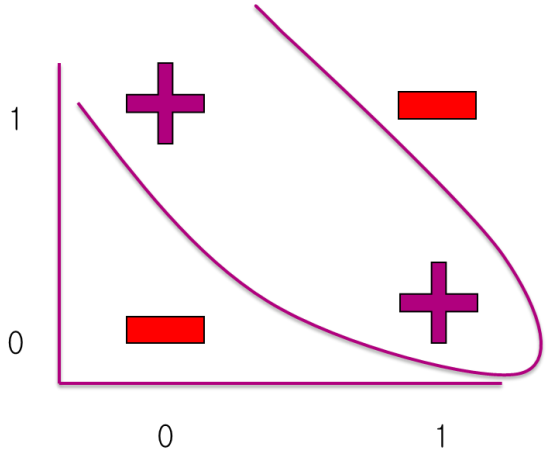


Linearly separable



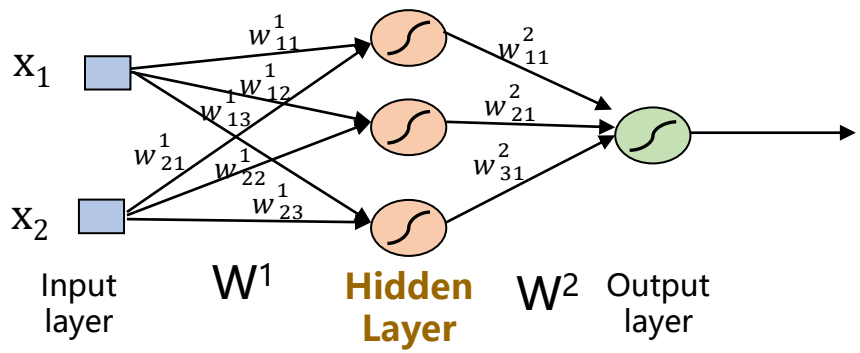
Not linearly separable

- **Solution: Non-linearity!**



Multi-layer Neural Network

- **Stacking hidden layers gives non-linearity**



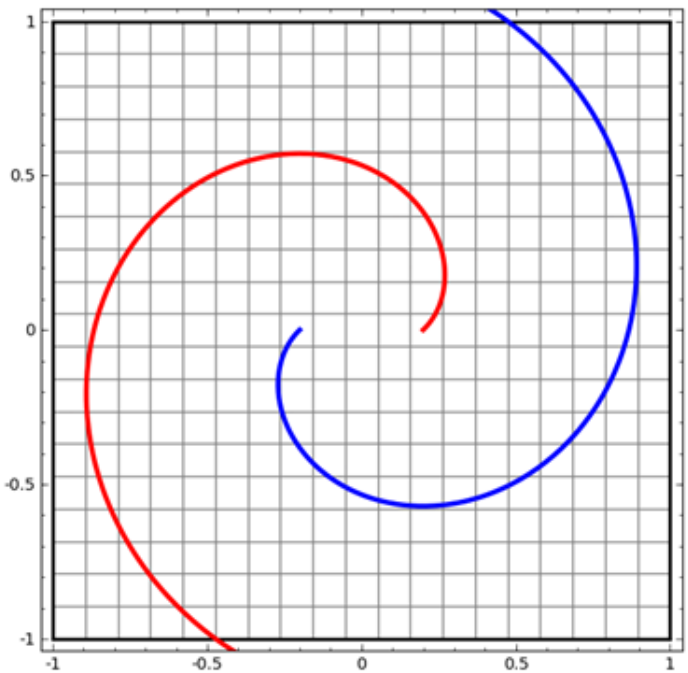
1st layer: $$(x_1 \quad x_2)\underbrace{\begin{pmatrix} w^1_{11} & w^1_{12} & w^1_{13} \\ w^1_{21} & w^1_{22} & w^1_{23} \end{pmatrix}}_{\mathbf{x}\cdot\mathbf{W}^1} = (z_1 \quad z_2 \quad z_3) \xrightarrow[\text{sigmoid}]{} (o_1 \quad o_2 \quad o_3)$$

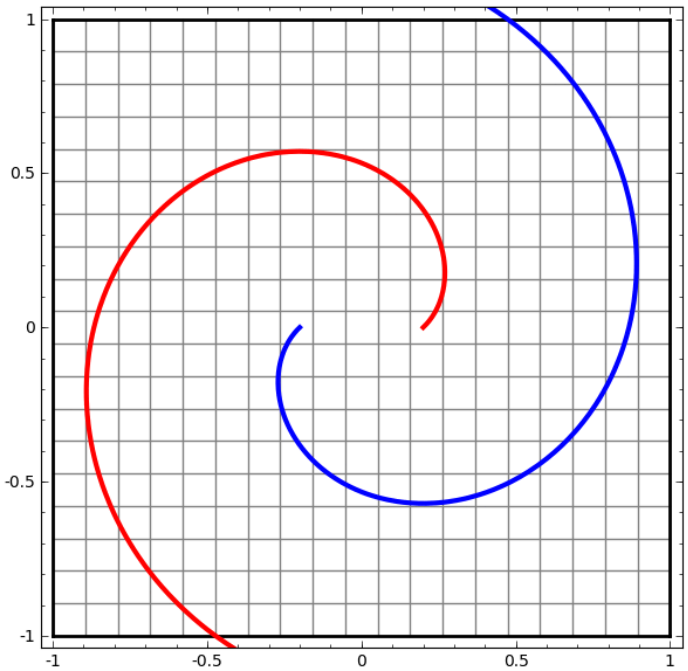
2nd layer: $$(o_1 \quad o_2 \quad o_3)\underbrace{\begin{pmatrix} w^2_{11} \\ w^2_{21} \\ w^2_{31} \end{pmatrix}}_{\mathbf{o}\cdot\mathbf{W}^2} = z \xrightarrow[\text{sigmoid}]{} [0,1]$$

- $\mathbf{x}\cdot\mathbf{W}^1$: linear transformation of input $\mathbf{x}$
- Sigmoid function: non-linear transformation
- 2nd layer: a logistic regression classifier

Multi-layer Neural Network

- **linear transform (xW) + non-linear transform (sigmoid)**

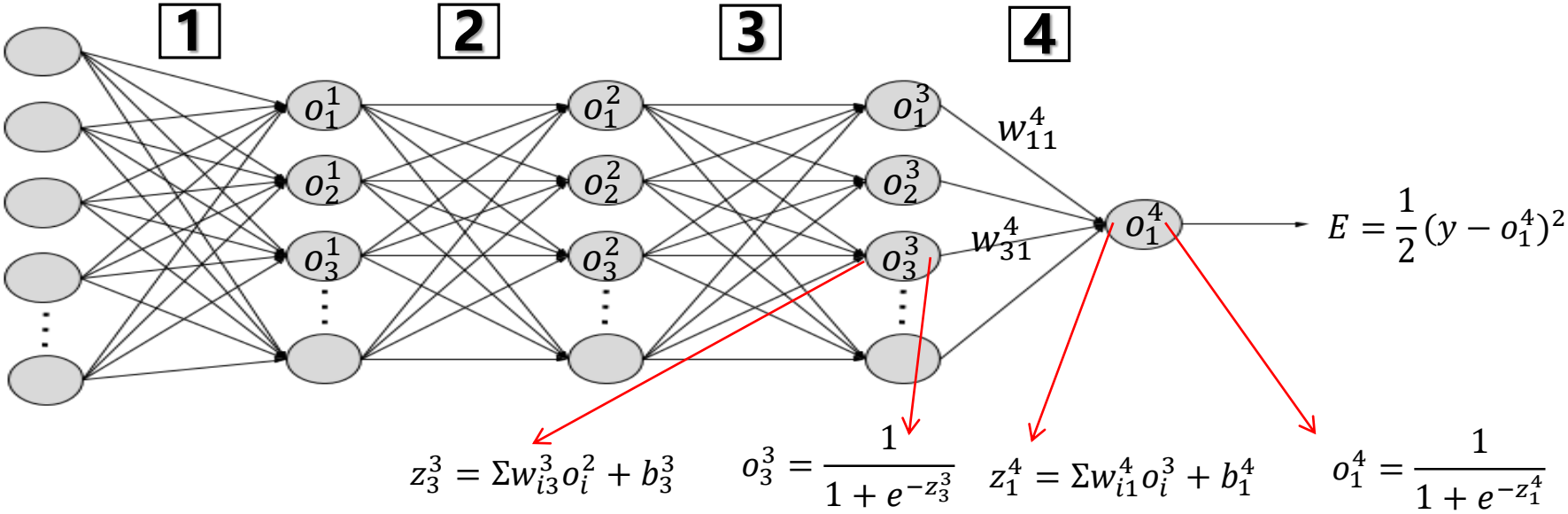




- **What hidden layers do**
  - They transform data from the previous layer to create a new data representation
  - They look for a good representation of data that can be classified better by a linear classification

Multi-layer Neural Network

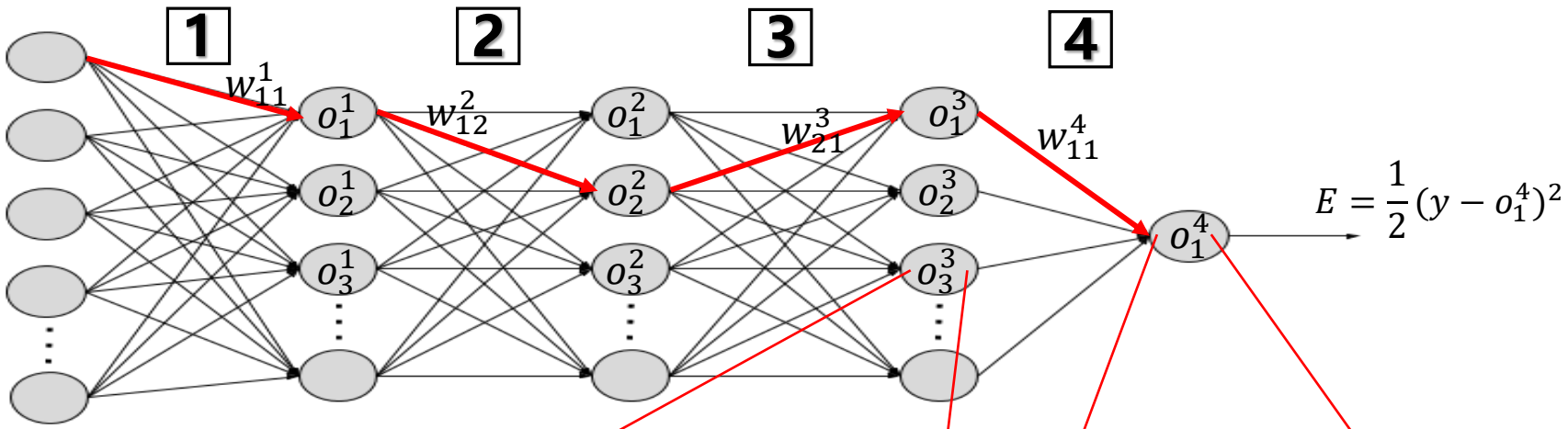
- **Training multi-layer neural networks**
  - Same solution: gradient descent!



General term: $z_j^p = \Sigma w_{ij}^p o_i^{p-1} + b_j^p$ $\quad o_j^p = \frac{1}{1 + e^{-z_j^p}}$

Multi-layer Neural Network

- **Computing partial derivative of the loss function E with respect to each weight is extremely complicated**



$$z_3^3 = \Sigma w_{i3}^3 o_i^2 + b_3^3 \qquad o_3^3 = \frac{1}{1+e^{-z_3^3}} \qquad z_1^4 = \Sigma w_{i1}^4 o_i^3 + b_1^4 \qquad o_1^4 = \frac{1}{1+e^{-z_1^4}}$$

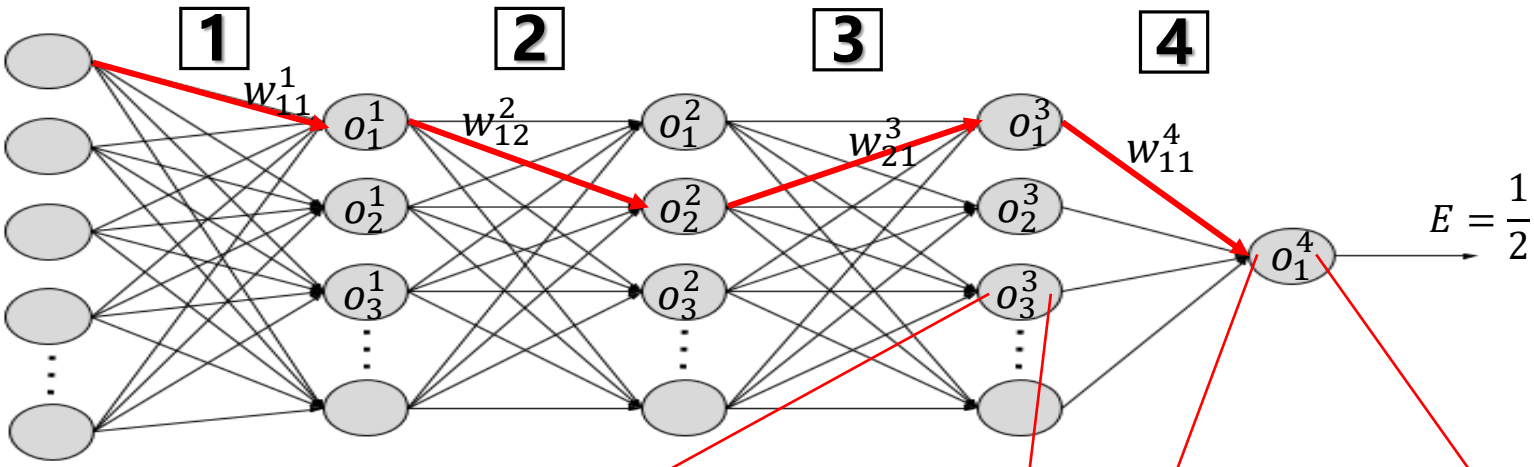
$$\frac{\partial E}{\partial w_{11}^4} = \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial w_{11}^4}$$

$$\frac{\partial E}{\partial w_{21}^3} = \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial o_1^3}\frac{\partial o_1^3}{\partial z_1^3}\frac{\partial z_1^3}{\partial w_{21}^3}$$

$$\frac{\partial E}{\partial w_{12}^2} = \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial o_1^3}\frac{\partial o_1^3}{\partial z_1^3}\frac{\partial z_1^3}{\partial o_2^2}\frac{\partial o_2^2}{\partial z_2^2}\frac{\partial z_2^2}{\partial w_{12}^2} + \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial o_2^3}\frac{\partial o_2^3}{\partial z_2^3}\frac{\partial z_2^3}{\partial o_2^2}\frac{\partial o_2^2}{\partial z_2^2}\frac{\partial z_2^2}{\partial w_{12}^2} + \cdots + \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial o_k^3}\frac{\partial o_k^3}{\partial z_k^3}\frac{\partial z_k^3}{\partial o_2^2}\frac{\partial o_2^2}{\partial z_2^2}\frac{\partial z_2^2}{\partial w_{12}^2}$$

Multi-layer Neural Network

- **Computing each partial derivative is extremely complicated**



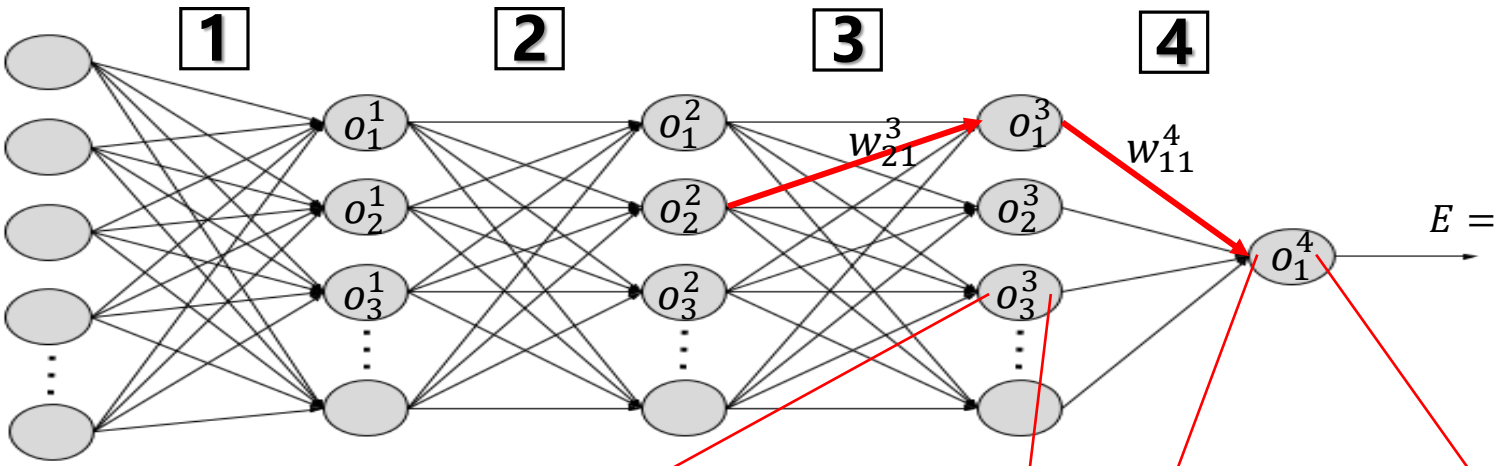
$$E = \frac{1}{2}(y - o_1^4)^2$$

$$z_3^3 = \Sigma w_{i3}^3 o_i^2 + b_3^3 \qquad o_3^3 = \frac{1}{1+e^{-z_3^3}} \qquad z_1^4 = \Sigma w_{i1}^4 o_i^3 + b_1^4 \qquad o_1^4 = \frac{1}{1+e^{-z_1^4}}$$

$$\frac{\partial E}{\partial w_{11}^1} =$$

We need an **efficient algorithm** to compute gradient for training deep neural networks..
=> **Back-propagation!**

Back-Propagation



$$E = \frac{1}{2}(y - o_1^4)^2$$

$$z_3^3 = \Sigma w_{i3}^3 o_i^2 + b_3^3 \qquad o_3^3 = \frac{1}{1 + e^{-z_3^3}} \qquad z_1^4 = \Sigma w_{i1}^4 o_i^3 + b_1^4 \qquad o_1^4 = \frac{1}{1 + e^{-z_1^4}}$$

$$\frac{\partial E}{\partial w_{11}^4} = \boxed{\frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}}\frac{\partial z_1^4}{\partial w_{11}^4} = \boldsymbol{-(y - o_1^4)o_1^4(1 - o_1^4)}o_1^3$$

$$\frac{\partial E}{\partial w_{21}^3} = \boxed{\frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}}\frac{\partial z_1^4}{\partial o_1^3}\frac{\partial o_1^3}{\partial z_1^3}\frac{\partial z_1^3}{\partial w_{21}^3}$$

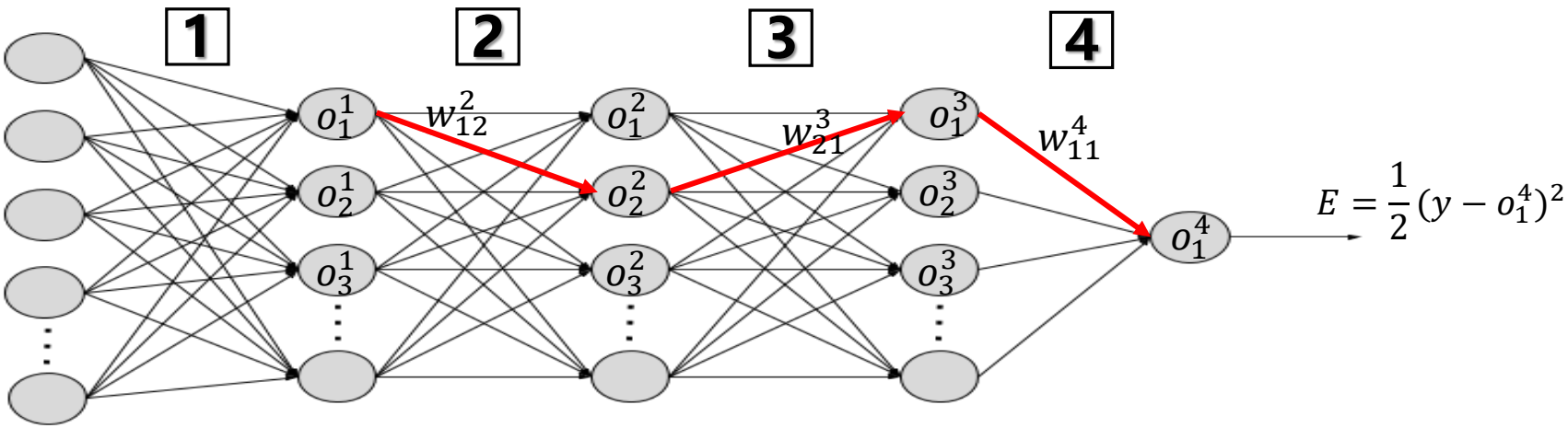
$$= \boldsymbol{-(y - o_1^4)o_1^4(1 - o_1^4)}\frac{\partial z_1^4}{\partial o_1^3}\frac{\partial o_1^3}{\partial z_1^3}\frac{\partial z_1^3}{\partial w_{21}^3}$$

$$= \boldsymbol{-(y - o_1^4)o_1^4(1 - o_1^4)}w_{11}^4 o_1^3(1 - o_1^3)o_2^2$$

$$\frac{\partial z_1^4}{\partial w_{11}^4} = o_1^3 \qquad \frac{\partial o_1^4}{\partial z_1^4} = o_1^4(1 - o_1^4) \qquad \frac{\partial E}{\partial o_1^4} = -(y - o_1^4)$$

$$\frac{\partial z_1^4}{\partial o_1^3} = w_{11}^4 \qquad \frac{\partial o_1^3}{\partial z_1^3} = o_1^3(1 - o_1^3) \qquad \frac{\partial z_1^3}{\partial w_{21}^3} = o_2^2$$

Back-Propagation



$$\frac{\partial E}{\partial w_{11}^4} = \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial w_{11}^4}$$

$$= -(y - o_1^4)o_1^4(1 - o_1^4)o_1^3$$

$$\frac{\partial E}{\partial w_{21}^3} = \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial o_1^3}\frac{\partial o_1^3}{\partial z_1^3}\frac{\partial z_1^3}{\partial w_{21}^3}$$

$$= -(y - o_1^4)o_1^4(1 - o_1^4)w_{11}^4 o_1^3(1 - o_1^3)o_2^2$$

$$\frac{\partial E}{\partial w_{12}^2} = \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial o_1^3}\frac{\partial o_1^3}{\partial z_1^3}\frac{\partial z_1^3}{\partial o_2^2}\frac{\partial o_2^2}{\partial z_2^2}\frac{\partial z_2^2}{\partial w_{12}^2} + \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial o_2^3}\frac{\partial o_2^3}{\partial z_2^3}\frac{\partial z_2^3}{\partial o_2^2}\frac{\partial o_2^2}{\partial z_2^2}\frac{\partial z_2^2}{\partial w_{12}^2} + \cdots + \frac{\partial E}{\partial o_1^4}\frac{\partial o_1^4}{\partial z_1^4}\frac{\partial z_1^4}{\partial o_k^3}\frac{\partial o_k^3}{\partial z_k^3}\frac{\partial z_k^3}{\partial o_2^2}\frac{\partial o_2^2}{\partial z_2^2}\frac{\partial z_2^2}{\partial w_{12}^2}$$

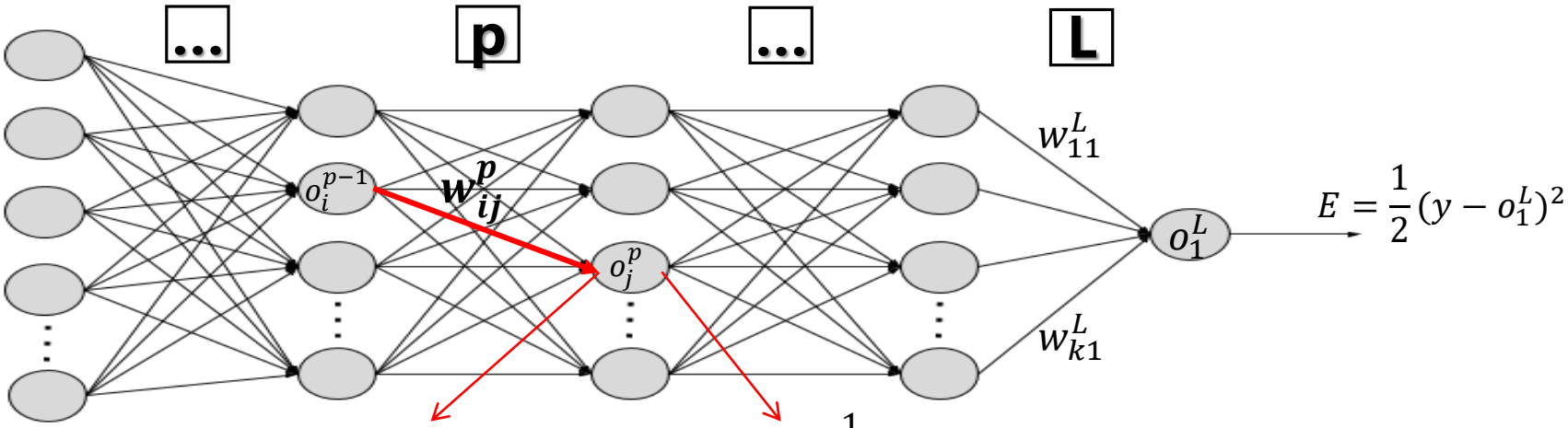
$$= -(y - o_1^4)o_1^4(1 - o_1^4)w_{11}^4 o_1^3(1 - o_1^3)w_{21}^3 o_2^2(1 - o_2^2)o_1^1$$

$$+ -(y - o_1^4)o_1^4(1 - o_1^4)w_{21}^4 o_2^3(1 - o_2^3)w_{22}^3 o_2^2(1 - o_2^2)o_1^1$$

$$+ -(y - o_1^4)o_1^4(1 - o_1^4)w_{k1}^4 o_k^3(1 - o_k^3)w_{2k}^3 o_2^2(1 - o_2^2)o_1^1$$

Back-Propagation

- **A general form: delta rule**



$$z_j^p = \Sigma w_{ij}^p o_i^{p-1} + b_j^p \qquad o_j^p = \frac{1}{1 + e^{-z_j^p}}$$

$$\frac{\partial E}{\partial w_{ij}^p} = \delta_j^p o_i^{p-1}$$

$$\text{where} \begin{cases} \delta_j^p = o_j^p \left(1 - o_j^p\right) \Sigma w_{jk}^{p+1} \delta_k^{p+1} & (p \neq L) \\ \delta_1^L = o_1^L (1 - o_1^L)(-(y - o_1^L)) & (p = L) \end{cases}$$

$$\frac{\partial E}{\partial w_{11}^L} = \frac{\partial E}{\partial o_1^L} \frac{\partial o_1^L}{\partial z_1^L} \frac{\partial z_1^L}{\partial w_{11}^L}$$

$$= -(y - o_1^L) o_1^L (1 - o_1^L)\, o_1^{L-1}$$

Back-Propagation

- **We can recursively compute delta from the output to input layer**

$$\frac{\partial E}{\partial w_{ij}^{p}} = \delta_j^p o_i^{p-1} \quad \text{where} \quad \begin{cases} \delta_j^p = o_j^p\left(1-o_j^p\right)\Sigma w_{jk}^{p+1}\delta_k^{p+1} & (p \neq L) \\ \delta_1^L = o_1^L(1-o_1^L)(-(y-o_1^L)) & (p = L) \end{cases}$$

$$\frac{\partial E}{\partial w_{ij}^{L}} = \delta_j^L o_i^{L-1}, \qquad \delta_j^L = \delta_1^L = o_1^L(1-o_1^L)(-(y-o_1^L)) \qquad (j=1)$$

$$\frac{\partial E}{\partial w_{ij}^{L-1}} = \delta_j^{L-1} o_i^{L-2}, \qquad \delta_j^{L-1} = o_j^{L-1}\left(1-o_j^{L-1}\right)\Sigma w_{jk}^{L}\delta_k^{L} \qquad (j = 1 \sim k_{L-1}) \quad (k = 1 \sim k_L,\ k_L = 1)$$

$$\frac{\partial E}{\partial w_{ij}^{L-2}} = \delta_j^{L-2} o_i^{L-3}, \qquad \delta_j^{L-2} = o_j^{L-2}\left(1-o_j^{L-2}\right)\Sigma w_{jk}^{L-1}\delta_k^{L-1} \qquad (j = 1 \sim k_{L-2}) \quad (k = 1 \sim k_{L-1})$$

$$\vdots$$

$$\frac{\partial E}{\partial w_{ij}^{1}} = \delta_j^1 x_i\ , \qquad \delta_j^1 = o_j^1\left(1-o_j^1\right)\Sigma w_{jk}^{2}\delta_k^{2} \qquad (j = 1 \sim k_1) \quad (k = 1 \sim k_2)$$

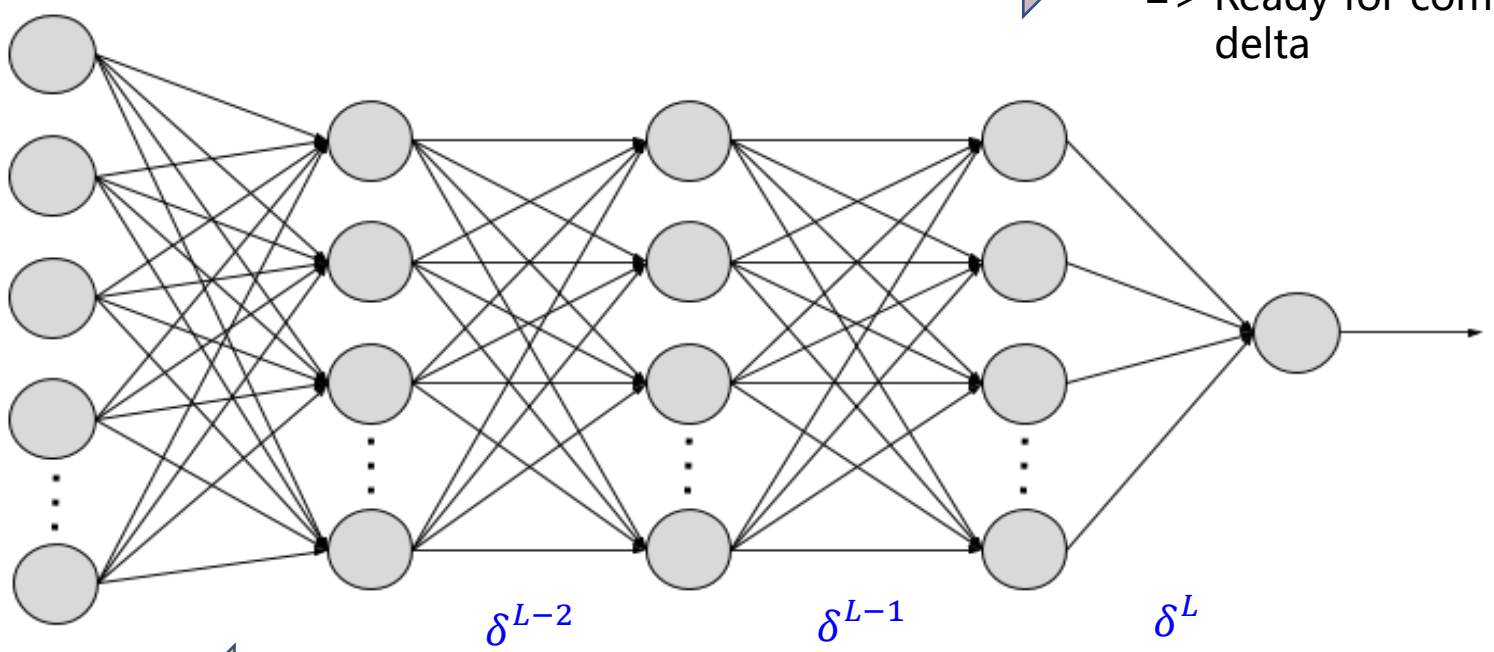
Back-Propagation

- **Back-propagation: an algorithm to compute gradient for deep neural networks very efficiently!**

Feed-forward

- Compute all o_j^p
- Compute $E = \frac{1}{2}(y - o_1^L)^2$

=> Ready for computing delta



Back-propagation

Back-Propagation

*gradient of the Logistic Regression model

$$\frac{\partial L}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N}[\, x_j \cdot (\sigma(s_i) - y_i)\,]$$

- **Algorithm**
  - For each training example, do:
    - **Feed-forward**: compute outputs for each neuron, o_j^p, compute $y - o_1^L$
    - **Back-propagation**: for each layer p (from **L to 1**), do:
      - Compute each $\frac{\partial E}{\partial w_{ij}^p} = \delta_j^p o_i^{p-1}$

$$\text{where } \begin{cases} \delta_j^p = o_j^p\left(1 - o_j^p\right)\Sigma w_{jk}^{p+1}\delta_k^{p+1} & (p \neq L) \\ \delta_1^L = o_1^L(1 - o_1^L)(-(y - o_1^L)) & (p = L) \end{cases}$$

      - Store each δ_j^p for further usage
  - **Cumulate:** compute average of $\frac{\partial E}{\partial w_{ij}^p}$ for all training data objects: n = [1 ~ N]

$$\frac{\partial E}{\partial w_{ij}^p} = \frac{1}{N}\sum_{n=1}^{N}\delta_j^p[n] \cdot o_i^{p-1}[n]$$

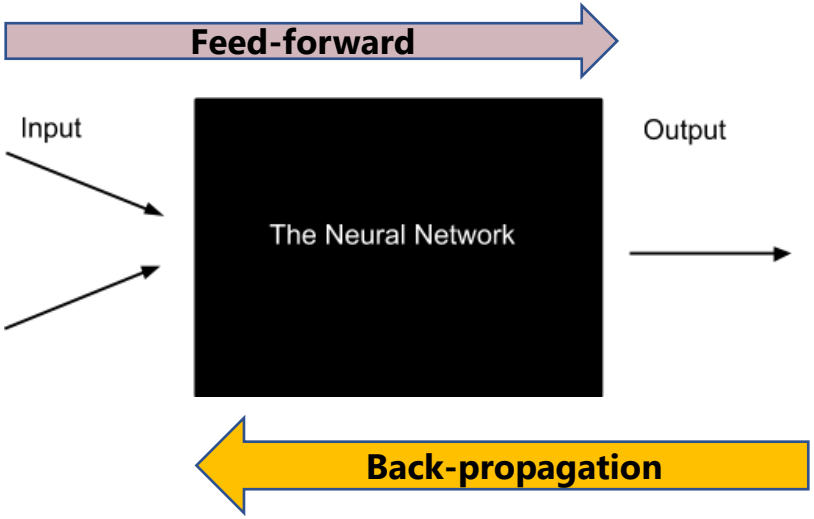
  - **Update**: Update all the weights (and biases) through $\mathbf{W}_{t+1} \leftarrow \mathbf{W}_t - \mu\nabla\mathbf{W}_t$ where $\mathbf{W}_t$ is the collection of all the weights and $\nabla\mathbf{W}_t$ is the gradient vector.

[Until converge]

$$\mathbf{W} = [\frac{\partial E}{\partial w_{11}^1}, \frac{\partial E}{\partial w_{12}^1}, \dots, \frac{\partial E}{\partial w_{k1}^L}]$$

Back-Propagation

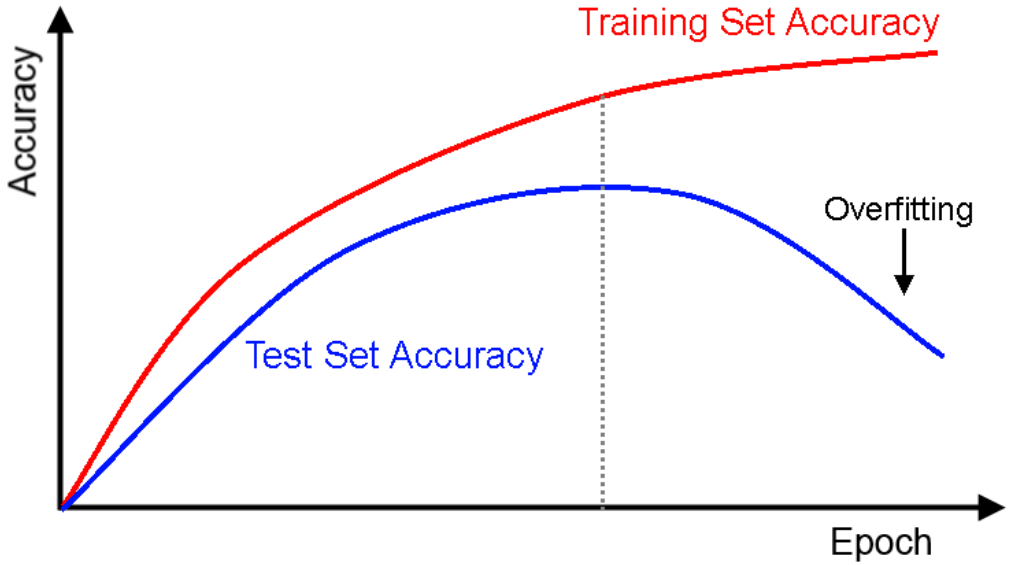
- **Various deep learning structures**
  - Fully-connected deep neural network
  - Autoencoder
  - Convolutional neural network (CNN)
  - Recurrent neural network (RNN)
  - Deep reinforcement learning
  - Generative Adversarial Networks (GAN)
  - Transformers, BERT, GTP and other deep NLP models
  - Etc….



- **Back-propagation is universally used for training any deep learning model**

Regularization

- **Overfitting: Perfect on training data, but poor on test data**
  - Bad generalization ability



- **Solutions to overfitting of neural networks**
  - L2 regularization
  - Dropout (skip)
  - Batch normalization (skip)

Regularization

- **L2 regularization**

A.K.A L2-norm

  - To add <u>the sum of the squares of the model parameters</u> $\boldsymbol{\Sigma(w_{ij})^2}$ multiplied with a regularization coefficient λ to the cost function.

$$E(\mathbf{W}) = \frac{1}{2}\sum_{d=1}^{n}(y_d - \hat{y}_d)^2 + \frac{\boldsymbol{\lambda}}{\mathbf{2}}\boldsymbol{\Sigma(w_{ij})^2}$$

  - Now we have to find optimal W* that minimizes the cost as well as the L2-norm
  - If the value of each w_i is small, then the decision boundary becomes smoother, thus reducing the overfitting.
  - λ controls the relative importance of **reducing the cost** VS **making model simple & smooth**

SGD Optimization with Minibatch

- **(deterministic) Gradient descent**
  - Compute average of $\frac{\partial E}{\partial w_{ij}^{p}}$ for **all training data objects**: n = [1 ~ N]

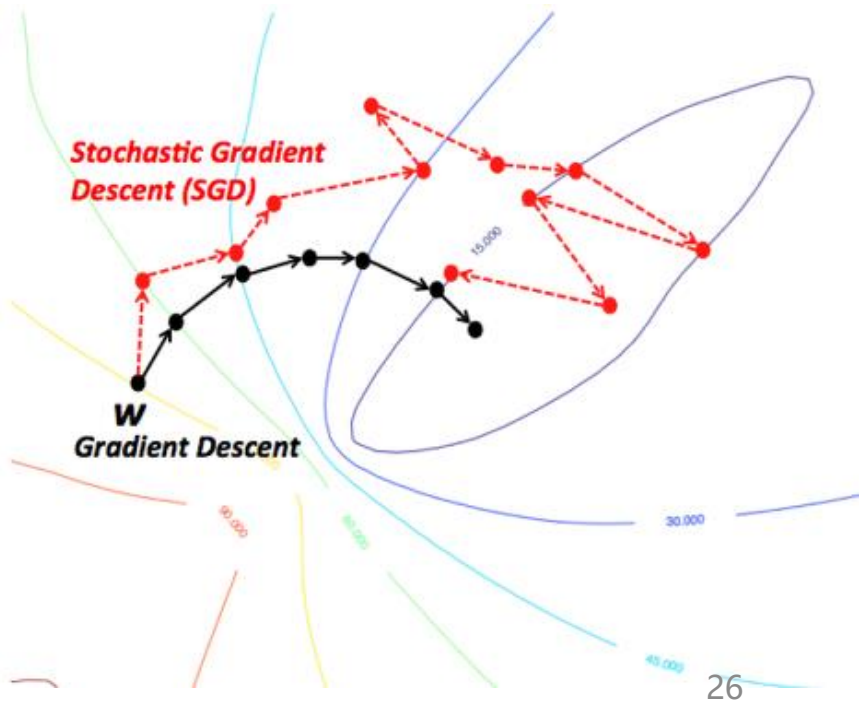
$$\frac{\partial E}{\partial w_{ij}^{p}} = \frac{1}{N}\sum_{n=1}^{N} \delta_j^p[n] \cdot o_i^{p-1}[n]$$

  - But, using the complete training set in each iteration can be very expensive and time consuming

- **Stochastic gradient descent**
  - Sample just one data randomly, compute cost, and update model

$$\frac{\partial E}{\partial w_{ij}^{p}} = \delta_j^p \cdot o_i^{p-1}$$

  - Looks fast, but unstable



SGD Optimization with Minibatch

- **Stochastic gradient descent with Minibatch**
  - Randomly sample **M** numbers of (T >> M) data instances
    - We call this mini-batch

$$\frac{\partial E}{\partial w_{ij}^{p}} = \frac{1}{\boldsymbol{M}} \sum_{n=1}^{\boldsymbol{M}} \delta_j^p[n] \cdot o_i^{p-1}[n]$$

  - It achieves the advantages of both the deterministic and stochastic gradient descent methods

Class Summary

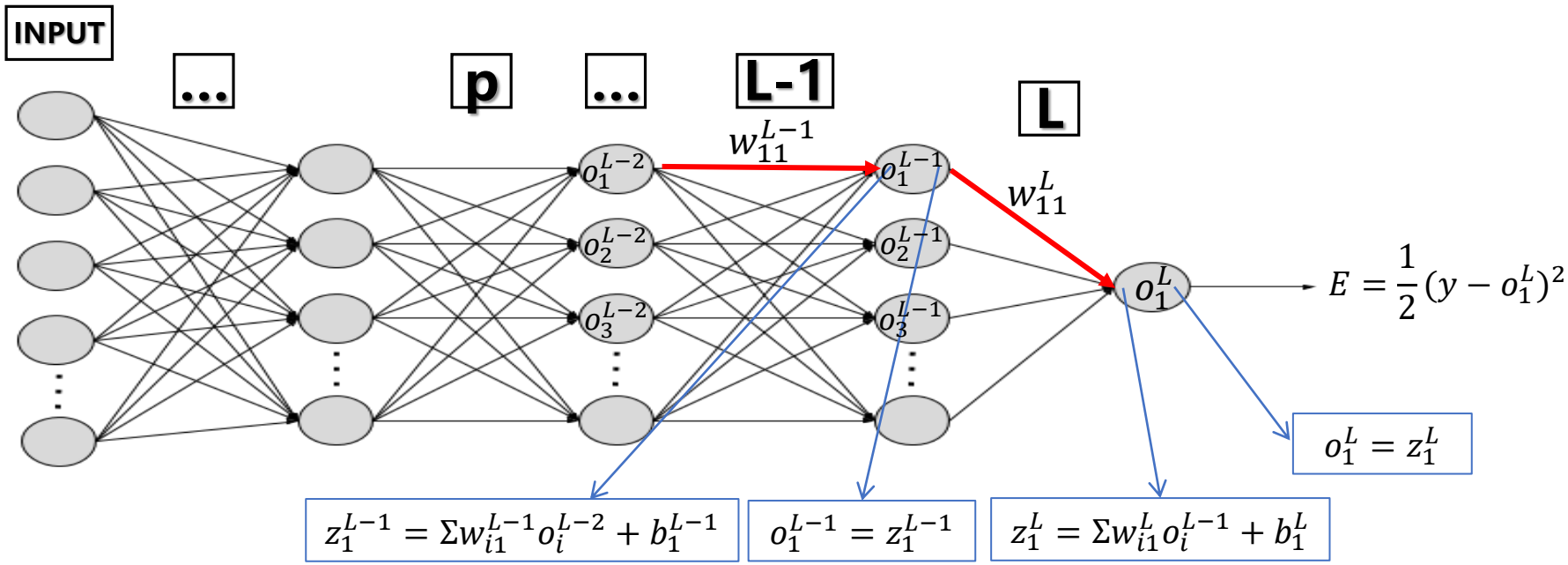
- **A perceptron**
- **Multi-layer Neural Network**
- **Back-propagation**
- **Several TMIs of neural networks, including…**
  - Gradient vanishing and ReLU
  - Overfitting and regularization skills
  - Gradient descent VS stochastic gradient descent
  - …..

Skipped Topics

- **Models:** CNN, RNN, LSTM, Autoencoder, Transformer, …
- **Applications:** Computer vision, natural language processing, …
- **Advanced optimizers:** AdaGrad, ADAM, RMSProp, …

Thank You

Back-Propagation



General term: $z_j^p = \Sigma w_{ij}^p o_i^{p-1} + b_j^p \qquad o_j^p = z_j^p$